

How AI Works

A beginner's journey inside the machine that learns

Study report | Updated August 4, 2026

Central question	How can a computer learn useful patterns without containing a human mind?
Core process	Examples → learning process → trained model → prediction
Course path	AI basics, data, training, neural networks, language models, a Python project, and responsible use.
Study features	Highlighted phrases, inline notes, editorial commentary, a pending-review item, reader notes, and a footnote.

Opening Report: Inside the Machine That Learns

Every day, artificial intelligence quietly helps people choose music, translate messages, find photographs, avoid spam, and answer questions. To a casual observer, these systems can seem almost magical. Behind the scenes, however, there is no tiny person thinking inside the computer. There are data, mathematical operations, learning algorithms, and many adjustable values¹ working together at remarkable speed.

EDITORIAL COMMENT

The opening moves from familiar uses of AI to the less visible machinery beneath them. This contrast introduces the course's central goal: replacing mystery with a clear account of learning from data.

This course follows the story of how an ordinary computer becomes a machine that can recognize patterns. We will begin by separating artificial intelligence from traditional rule-based software. Then we will enter the training room, where a model studies examples, makes predictions, measures its mistakes, and gradually improves. From there, we will explore neural networks, discover how language models learn relationships between words, and examine why even powerful AI can produce confident but incorrect answers.

Our investigation unfolds in seven stages: what AI is; learning from data; training a model; neural networks; tokens, context, and next-token prediction²; building a small AI in Python; and responsible use. The final stage considers bias, privacy, reliability, and human judgment.

By the end, AI should feel less like magic and more like an understandable—though still impressive—engineering achievement. Each lesson includes a plain-language explanation, a familiar example, a quick question, and, when useful, a small experiment.

Lesson 1: What Is Artificial Intelligence?

Artificial intelligence is the broad name for computer systems designed to perform tasks that appear to require human abilities[?], such as recognizing speech, interpreting images, finding patterns, making predictions, or generating language.

Not every useful computer program is AI. A calculator follows precise rules written by a programmer. A machine-learning spam filter is different: it studies examples of spam and ordinary email, detects patterns in those examples, and uses the patterns to classify new messages.

The basic process is simple to describe: examples enter a learning process; the learning process produces a trained model³; and the trained model makes a prediction about new information.

Quick Check

Which system learns from data—a light switch, a calculator following fixed arithmetic rules, or an email filter trained on examples of spam? Answer: the email filter, because it learns patterns from labeled examples.

Your turn: Name one AI system you encounter in daily life. What information or examples do you think it learned from?

Lesson 2: How Machines Learn from Data

A machine-learning system **learns from examples rather than receiving every rule directly**. Each example contains information the model can examine. The model searches across many examples for relationships that help it make a useful prediction.

Suppose we want a model to distinguish apples from oranges. We might give it examples described by color, weight, texture, and shape. These measurable properties are called **features**⁴. The correct answer attached to each example—apple or orange—is called its **label**⁵.

During training, the **model studies the features and labels together**. At first, its predictions may be poor. A learning algorithm measures the errors and adjusts the model's parameters so that later predictions are more accurate. Training is a repeated cycle: predict, compare, adjust, and try again.

Good performance on remembered examples is not enough. We normally divide the available data into a training set and a test set. The model learns from the training set, but the test set contains **separate examples that were not used to adjust the model**. Testing helps reveal whether the model learned a general pattern or merely memorized its lessons.

Data quality matters as much as quantity[?]. If the examples are incorrect, incomplete, or unrepresentative, the model may learn misleading patterns. A fruit classifier trained only on green apples might wrongly treat redness as evidence that a fruit is an orange. The data should **represent the real situations in which the model will be used**.

Quick Check

A model scores perfectly on its training examples but performs badly on new examples. Did it necessarily learn a useful general pattern? No. It may have memorized the training data, a problem known as **overfitting**⁶.

Your turn: Imagine training an AI system to identify whether a photograph was taken indoors or outdoors. Name three useful features and one possible source of bias in the training data.

Lesson 3: How Training Improves a Model

Training **turns mistakes into directions for improvement**. A model receives an input, makes a prediction, and compares that prediction with the known answer. The difference tells the training process how far the model is from the desired result.

Imagine a model predicting the price of a house. If the correct price is \$300,000 and the model predicts \$240,000, its error is \$60,000. A **loss function**⁷ **converts the difference between predictions and answers into a number**. A larger loss usually means that the prediction was worse.

The learning algorithm then asks which internal parameters contributed to the error. In a large model, millions or even billions of parameters may share responsibility. The algorithm **calculates a small adjustment to many parameters**, aiming to make the loss slightly lower the next time similar data appears.

One widely used adjustment method is **gradient descent**⁸. Imagine standing on a foggy hillside and trying to reach the lowest point. You cannot see the whole landscape, but you can feel which direction slopes downward. By taking repeated downhill steps, you may approach a low point[?]. Gradient descent similarly uses the local slope of the loss to choose a direction for updating parameters.

The size of each update is controlled by the learning rate⁹. If the learning rate is too small, training may be extremely slow. If it is too large, the model may repeatedly jump past a useful solution. Effective training requires a balance between learning too slowly and overshooting.

One complete pass through the training data is called an epoch. Models commonly train for many epochs, but more is not always better. We can monitor performance on a validation set while training. If training loss continues to improve while validation performance becomes worse, the model may be starting to overfit.

Quick Check

Why not make the learning rate as large as possible? A very large update can skip over better parameter settings or make training unstable.

Your turn: Picture yourself walking downhill in fog. What do the hill's height, downhill direction, step size, and repeated steps represent in model training?

Lesson 4: Neural Networks

A neural network is a model built from layers of simple mathematical units often called artificial neurons¹⁰. The name is loosely inspired by biological brains, but an artificial neural network is an engineered system of calculations, not a miniature human brain.

Each artificial neuron receives several input numbers. It multiplies each input by a learned weight, adds the results together, and usually adds another adjustable value called a bias term. An activation function¹¹ then transforms the total and determines the value passed forward.

The weights tell the network which inputs deserve more or less influence. Consider a simple system estimating whether a student will complete an assignment. Time available, assignment length, and previous completion patterns might enter as numbers. Training adjusts their weights according to how strongly each input helps predict the answer.

Neurons are arranged in layers. The input layer receives the original information. One or more hidden layers transform it into increasingly useful internal patterns. The output layer produces the final prediction. Information moving from input to output is called a forward pass.

Hidden layers allow a network to build representations in stages. In an image system, early layers may respond to edges or colors, middle layers may combine them into textures and shapes, and later layers may respond to larger structures. Training helps the network discover useful combinations without a separate programmed rule for every possible shape.

After a forward pass, the loss function measures prediction error. Backpropagation¹² works backward through the network to calculate how each weight contributed to that error. Gradient descent then uses those calculations to update the weights.

Deep learning uses neural networks with multiple hidden layers. Greater depth can help represent complicated patterns, but a larger network is not automatically better. It may require more data, computation, careful evaluation, and safeguards against overfitting.

Quick Check

Does an artificial neuron understand its inputs in the human sense? No. It performs numerical operations; useful behavior emerges from many trained operations working together.

Your turn: Draw a tiny network with three input circles, two hidden-layer circles, and one output circle. What real-world quantities could the three inputs represent, and what could the output predict?

Искусственный интеллект учится на примерах: система сравнивает свои прогнозы с известными ответами, замечает ошибки и постепенно изменяет внутренние параметры. Поэтому качество модели зависит не только от алгоритма, но и от данных, целей обучения и человеческой проверки.*

Editor's footnote * — This independent Russian paragraph restates the training cycle: compare predictions with known answers, measure errors, and adjust parameters. It also emphasizes data quality and human review.

Inline Notes and Pending Review

Marker	Inline note
1	Adjustable values are the model's parameters.
2	Next-token prediction estimates the next unit of text from preceding context.
3	A trained model is the result of learning from examples and can be used on new input.
4	Features are measurable properties or inputs used to make a prediction.
5	A label is the known answer associated with a training example.
6	Overfitting means learning the training examples too narrowly to generalize well.
7	A loss function measures prediction error as a number training can minimize.
8	Gradient descent updates parameters in a direction that reduces loss.
9	The learning rate controls the size of each parameter update.
10	Artificial neurons are mathematical units that combine and transform input values.
11	An activation function transforms a neuron's total before passing it onward.
12	Backpropagation calculates how network parameters contributed to prediction error.
?	Pending review: consider whether "imitate aspects of human abilities" would be more precise.
?	Pending clarification: gradient descent is not guaranteed to find the best possible solution.

Reader's Notes

Reader's note	The contrast between a calculator and a spam filter is useful. One follows rules supplied directly by a programmer; the other derives classification patterns from examples.
Reader's question	If a model learns biased patterns from its training data, which parts of training and evaluation can reveal or reduce that bias?
Lesson 2 note	The apple-and-orange example makes features and labels concrete, while the separate test set explains why memorization is not the same as learning a general pattern.
Lesson 3 note	The foggy-hillside analogy connects loss, gradient direction, learning rate, and repeated updates without requiring calculus.
Lesson 4 note	A neural network is a system of layered numerical transformations. The brain analogy is historical and limited.

Course Vocabulary

Term	Meaning in this course
artificial intelligence	Computer systems designed to perform tasks associated with human intelligence.
machine learning	Learning patterns from data rather than relying only on fixed rules.
model	A trained mathematical system that produces predictions or generated output.
parameter	An adjustable numerical value changed during training.
neural network	A model made from connected layers of mathematical operations.
token	A unit of text processed by a language model.
bias	A systematic tendency that can produce uneven or unfair results.
feature	A measurable property or input used by a model to make a prediction.
label	The known answer associated with a training example.
training set	The examples used to adjust a model during training.
test set	Separate examples used to evaluate a model on unseen data.
overfitting	Learning training examples too narrowly and performing poorly on new examples.
loss function	A rule that converts prediction error into a number for training to minimize.
gradient descent	A method for adjusting parameters in a direction that reduces loss.
learning rate	The setting that controls the size of each parameter update.
epoch	One complete pass through the training data.
validation set	Separate data used during training to monitor generalization.
artificial neuron	A mathematical unit that combines input values and passes a transformed value forward.
activation function	A function that transforms a neuron's combined input.
hidden layer	An internal layer that transforms information into learned representations.
forward pass	The movement and transformation of information from input to output.
backpropagation	A method for calculating how network parameters contributed to error.
deep learning	Machine learning using neural networks with multiple hidden layers.

This course was created by Codex, an AI coding agent from OpenAI.